

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 1

Grundlagen der Kommunikation im Web

Bevor in den kommenden Vorlesungen die serverseitige Programmierung mit PHP behandelt wird, ist es wichtig zu verstehen, was genau passiert, wenn in einem Webbrowser eine Adresse wie <https://www.fh-ooe.at/> eingetippt und die Eingabetaste gedrückt wird. Das Ergebnis ist bekannt: Nach kurzer Zeit erscheint eine fertig gerenderte Webseite. Zwischen Tastendruck und fertig geladener Seite läuft jedoch eine erstaunlich lange Kette an Vorgängen ab, an der eine Vielzahl von Protokollen, Diensten und Rechnern beteiligt ist. Die Kenntnis dieser Vorgänge ist für die Arbeit von Webdeveloper*innen wesentlich, da sich viele Eigenheiten von Dingen wie HTTP, Sessions, Cookies, Formularen oder APIs (mehr dazu in späteren Vorlesungen) direkt daraus ableiten lassen.

Dieses Kapitel nimmt einen konkreten Webrequest als Leitmotiv und arbeitet entlang dieses Beispiels die wichtigsten Netzwerk- und Protokoll-Grundlagen auf. Ziel ist ein solides Verständnis des *Weges*, den ein Request vom Browser zum Webserver und wieder zurück nimmt. Dieses Verständnis bildet die Grundlage für alle weiteren Kapitel, in denen es um das geht, was am Webserver selbst geschieht: das Verarbeiten des Requests mit PHP und das Erzeugen einer Antwort.

1.1 Zur Web Technologies Vorlesung

Die Vorlesung Web Technologies – kurz WET2VO – baut auf der Vorlesung Web Fundamentals (WEF1VO) direkt auf. Die Modalitäten sind daher weitgehend gleich. Im Folgenden sind die wichtigsten organisatorischen und inhaltlichen Dinge jedoch der Vollständigkeit halber angeführt.

1.1.1 Organisatorisches

Web Technologies behandelt serverseitige Webentwicklung und ist auf *zwölf Termine zu zwei Einheiten* aufgeteilt. Die Beurteilung erfolgt im Rahmen einer *elektronischen Klausur*, die am Ende des Semesters stattfindet. Die Klausur dauert 90 Minuten und findet ohne Zuhilfenahme der Unterlagen statt.

Die dazugehörigen Übungen sind auf je *zwölf Termine zu zwei Einheiten* aufgeteilt. Dabei werden Übungsaufgaben in Teams durchgeführt, von den zwölf Terminen beinhalten sechs eine Abgabe, die anderen sechs sind code-along Übungen.

Die Unterlagen zum Kurs befinden sich auf der eLearning-Plattform der FH Oberösterreich in den zugehörigen Kursen für Vorlesung¹ und Übung².

Zum Modul Web Technologies (WEB2) gehört weiters noch die Lehrveranstaltung Relational Databases (RDB2VO und RDB2UE). Diese beschäftigt sich mit der Persistierung, d. h. Datenbanken. Diese Inhalte werden in der zweiten Hälfte dieser Lehrveranstaltung integriert, wenn etwa eine Datenbank zur Persistierung von Inhalten benötigt wird.

1.1.2 Inhalte der Vorlesung

In diesem Semester liegt der Schwerpunkt auf serverseitiger Webentwicklung, vornehmlich mit PHP. Dabei werden folgende Themengebiete behandelt:

- Grundlagen der Kommunikation im Web (dieses Kapitel),
- PHP-Sprachgrundlagen (Syntax, Datentypen, Bibliotheksfunktionen),
- Regular Expressions,
- objektorientiertes PHP,
- Komponenten und Standards,
- Routing und Template-Systeme,
- Formularverarbeitung,
- Sessions und Authentifizierung,
- Datum, Zeit und Internationalisierung,
- Medienverarbeitung,
- REST APIs und AJAX, sowie
- Microframeworks und Testing.

1.2 Der Webrequest

Als Leitmotiv für die Vorlesung dient folgendes Szenario: Ein Laptop wird mit dem WLAN verbunden, im Browser wird die Adresse <https://www.fh-ooe.at/> eingetippt und die Eingabetaste gedrückt. Kurz darauf erscheint die Startseite der FH Oberösterreich. Was in dieser kurzen Zeitspanne im Hintergrund geschieht, lässt sich in eine Reihe wohldefinierter Schritte zerlegen.

Abbildung 1.1 zeigt eine stark vereinfachte Sicht auf die beteiligten Komponenten. Tatsächlich sind mindestens folgende Schritte nötig, damit die Seite im Browser erscheint – sie bilden den roten Faden dieser Vorlesung:

1. Der Browser muss verstehen, *was* angefordert wurde. Dazu wird die eingetippte Adresse (der *URL*) zerlegt (Abschnitt 1.3).
2. Der Laptop braucht eine eigene IP-Adresse im lokalen Netz. Diese bekommt er automatisch über *DHCP* (Abschnitt 1.7.4).
3. Der Name *www.fh-ooe.at* muss in eine IP-Adresse übersetzt werden. Dafür wird das *DNS* befragt (Abschnitt 1.6).

¹<https://elearning.fh-ooe.at/course/view.php?id=43705>

²<https://elearning.fh-ooe.at/course/view.php?id=43703>

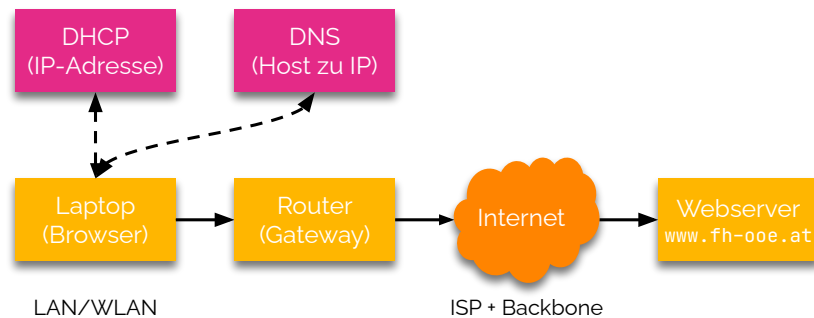


Abbildung 1.1: Vereinfachte Sicht auf die beteiligten Komponenten eines Webrequests. Neben Client und Server sind Hilfsdienste wie DHCP (für die lokale Netzwerkkonfiguration) und DNS (für die Namensauflösung) erforderlich, ebenso der lokale Router als Übergang ins Internet.

4. Für die lokale Zustellung an den Router muss die zugehörige Hardware-Adresse (*MAC*) ermittelt werden – das geschieht mit *ARP* (Abschnitt 1.7.5).
5. Der Browser baut zu Port 443 (*HTTPS*) des Webserver eine *TCP*-Verbindung auf (Abschnitt 1.8) und
6. verhandelt darüber per *TLS* einen verschlüsselten Kanal (Abschnitt 1.9).
7. Über diesen Kanal wird schließlich der eigentliche *HTTP-Request* verschickt (Abschnitt 1.10).
8. Der Webserver verarbeitet den Request und liefert eine *HTTP-Response* zurück, die *HTML*-Markup enthält – oder diesen Markup-Code dynamisch durch eine Skriptsprache wie *PHP* erzeugen lässt.
9. Der Browser parst das *HTML*, lädt eingebettete Ressourcen nach und rendert die Seite. Dieser letzte Schritt war bereits Thema von Web Fundamentals im ersten Semester.

Bei den Schritten 5 bis 8 läuft im Hintergrund jeweils noch *Encapsulation* ab: Die Daten werden vom sendenden Rechner Schicht für Schicht in *TCP*-, *IP*- und *Ethernet*-Header verpackt und vom empfangenden Rechner in umgekehrter Reihenfolge wieder ausgepackt (siehe Abschnitt 1.5.3). Dieser Vorgang wiederholt sich für jedes einzelne Paket – bei einem typischen Webrequest also viele Hundert Male.

All diese Schritte geschehen in Bruchteilen einer Sekunde. Jede einzelne Stufe setzt auf mehreren Protokollen auf, die wiederum in einem Schichtenmodell organisiert sind. Die nächsten Abschnitte arbeiten diese Schritte der Reihe nach ab.

1.3 Der URL als Adressierungsschema

Ein *URL* (Uniform Resource Locator) ist eine standardisierte Zeichenkette, mit dem eine Ressource im Web eindeutig adressiert werden kann. Der URL wurde historisch in RFC 1738 [3] definiert und später durch die allgemeinere URI-Spezifikation in RFC 3986 [2] abgelöst bzw. erweitert. Parallel dazu pflegt die WHATWG einen URL Living Standard [11], der die IETF-Dokumente vereinheitlicht und laufend an das tatsächliche

Tabelle 1.1: Einige gebräuchliche URL-Schemata.

Schema	Bedeutung
http	Hypertext Transfer Protocol (unverschlüsselt)
https	Hypertext Transfer Protocol Secure (verschlüsselt)
file	Lokale Datei im Dateisystem
mailto	E-Mail-Adresse
ftp	File Transfer Protocol
ws/wss	WebSocket (unverschlüsselt/verschlüsselt)

Verhalten moderner Browser anpasst. Im allgemeinsten Fall sieht ein URL so aus:

`<schema>:<schemaspezifischer Teil>`

Der Schemaname am Anfang legt fest, wie der restliche Teil zu interpretieren ist. Tabelle 1.1 zeigt einige gebräuchliche Schemata.

Nicht jedes Schema hat zwangsläufig eine Kommunikation über ein Netzwerk zur Folge. Ein URL vom Typ `file://` etwa öffnet eine Datei direkt im lokalen Dateisystem, *ohne* dass ein Webserver und ein Request beteiligt sind. Wird auf diese Weise eine PHP-Datei geöffnet, so sieht man im Browser lediglich den reinen PHP-Quelltext – es findet keine Verarbeitung statt, weil keine PHP-Engine involviert ist. Mehr dazu in Vorlesung 2. Ab jetzt sind daher primär `http`- und `https`-URLs interessant.

1.3.1 Aufbau eines HTTP-URLs

Ein HTTP-URL hat viele mögliche Elemente. Das folgende (etwas konstruierte) Beispiel zeigt alle möglichen Bestandteile:

`https://user:pass@www.example.org:8080/shop/item.php?id=42&lang=de#top`

Die Einzelteile haben dabei folgende Bedeutung:

https:// Das Schema, hier HTTP über TLS (verschlüsselt).

user:pass Optionale Zugangsdaten für HTTP-Basic-Auth (`.htaccess`). In der Praxis kaum mehr verwendet und aus Sicherheitsgründen abzulehnen, da Passwörter im Klartext im URL stehen.

www.example.org Der *Host*. Das kann ein *Fully Qualified Domain Name* (FQDN, siehe Abschnitt 1.6.1), oder eine IPv4- oder IPv6-Adresse sein (Abschnitt 1.7.1).

:8080 Der *Port*, auf dem der Webserver auf Verbindungen hört. Wird er nicht angegeben, verwendet der Browser den jeweiligen Standardport (80 für HTTP, 443 für HTTPS).

/shop/item.php Der *Pfad* (*url-path*) der angeforderten Ressource. Aus Server-Sicht ist das zunächst einmal einfach ein Name, der intern auf eine Datei, ein Skript oder einen Routing-Eintrag abgebildet wird.

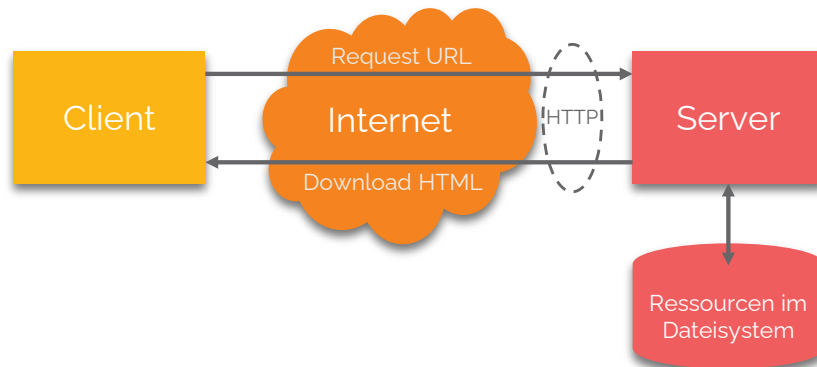


Abbildung 1.2: Client-Server-Kommunikation über das Internet.

?id=42&lang=de Der *Query-String*. Er beginnt mit ? und enthält ein oder mehrere **name=wert**-Paare, die in weiterer Folge durch & voneinander getrennt sind. Über den Query-String werden Parameter an den Server übergeben.

#top Das *Fragment*. Es beginnt mit # und adressiert einen bestimmten Abschnitt innerhalb der Ressource, etwa einen HTML-Anker. Wichtig: Das Fragment wird *nicht* mit an den Server geschickt, sondern ausschließlich clientseitig ausgewertet.

Für das am Anfang des Abschnitts vorgestellte Beispiel ist der URL wesentlich einfacher: `https://www.fh-ooe.at/` besteht aus Schema `https`, Host `www.fh-ooe.at`, implizitem Port 443 und als Pfad das Wurzel-Verzeichnis `/`. Query und Fragment sind nicht vorhanden.

All diese Informationen muss der Browser als Erstes aus der eingetippten Zeichenkette extrahieren, bevor er irgendetwas anderes tun kann.

1.4 Das Client-Server-Modell

Das Web ist, wie sehr viele andere Netzwerkanwendungen auch, nach dem *Client-Server-Modell* aufgebaut. Ein Rechner übernimmt die Rolle des *Servers* und stellt einen bestimmten Dienst bereit. Andere Rechner – die *Clients* – nutzen diesen Dienst, indem sie dem Server Anfragen schicken. Der Server beantwortet diese Anfragen und wartet ansonsten passiv auf weitere Clients. Abbildung 1.2 – bekannt aus dem 1. Semester – stellt diesen Zusammenhang dar.

Im Web ist der Client typischerweise ein Webbrowser (Chrome, Firefox, Safari etc.), der Server ein sogenannter *Webserver*. Der Begriff Webserver ist dabei zweideutig: Er bezeichnet sowohl den physischen oder virtuellen Rechner als auch die Software, die auf diesem Rechner läuft und eingehende HTTP-Requests verarbeitet. Gängige Webserver-Softwareprodukte sind *nginx*³, der *Apache HTTP Server*⁴ und *LiteSpeed*⁵. Erster hält Anfang April 2026 dabei bei einem Marktanteil von 32,8 % (2025: 33,8 %, 2024: 34,1 %, 2023: 34,3 %), zweiter bei 24,0 % (2025: 26,8 %, 2024: 30,2 %, 2023: 32,3 %), und

³<https://nginx.org/>

⁴<https://httpd.apache.org/>

⁵<https://www.litespeedtech.com/>

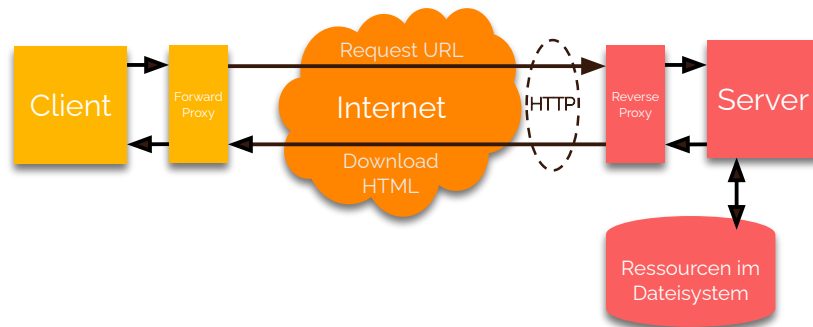


Abbildung 1.3: Client-Server-Kommunikation mit Forward und Reverse Proxies im Kommunikationsablauf.

dritter bei 15,2 % (2025: 14,5 %, 2024: 13,1 %, 2023: 11,6 %) [10]. Ein Sonderfall sind Anbieter wie Cloudflare, die als Reverse-Proxy vor der eigentlichen Infrastruktur stehen (siehe Abschnitt 1.4.1). In der verlinkten Statistik werden sie als eigene Einträge geführt, verbergen aber den dahinterliegenden tatsächlichen Webserver.

Der Webserver alleine kann zunächst nur statische Dateien ausliefern – also HTML-Dokumente, CSS-Dateien, Bilder, JavaScript-Dateien und Ähnliches. Für dynamisch erzeugte Inhalte wird eine zusätzliche Komponente benötigt, die den Request annimmt, mit Hilfe einer Skriptsprache wie PHP verarbeitet und das Ergebnis (z. B. HTML) an den Webserver zurückgibt, damit dieser es als Response an den Client zurückschicken kann. Vorlesung 2 wird darauf genauer eingehen.

1.4.1 Proxies, Reverse Proxies und Firewalls

In der Praxis ist die Darstellung „ein Client spricht direkt mit einem Webserver“ oft zu einfach. Gerade bei größeren Webanwendungen befinden sich zwischen Client und eigentlichem Anwendungsserver noch weitere Zwischenstationen, die bestimmte Aufgaben übernehmen. Zwei Begriffe sind hier besonders wichtig:

Forward Proxy Ein *Forward Proxy* (kurz: Proxy) steht *im Namen des Clients* zwischen diesem und dem Internet. Der Browser schickt seine Requests nicht direkt an den Zielservers, sondern an den Proxy, der die Anfrage stellvertretend weiterleitet. Typisch ist dieser Aufbau in Firmennetzen, wo ein Proxy zentral alle ausgehenden Verbindungen zwischenspeichert (*Caching*), protokolliert oder nach Inhalten filtert.

Reverse Proxy Ein *Reverse Proxy* steht *im Namen des Servers* zwischen dem Internet und dem eigentlichen Anwendungsserver. Der Client merkt davon nichts – aus seiner Sicht spricht er ganz normal mit „dem Webserver“. Tatsächlich nimmt der Reverse Proxy den Request entgegen und reicht ihn intern an einen oder mehrere dahinterliegende Anwendungsserver weiter.

Abbildung 1.3 stellt dar, wo die Proxies im Kommunikationsablauf positioniert sind.

Reverse Proxies sind aus modernen Webdeployments kaum mehr wegzudenken, weil sie eine Reihe wichtiger Aufgaben übernehmen: Sie terminieren die TLS-Verschlüsselung (siehe Abschnitt 1.9), verteilen die Last auf mehrere Backend-Server (*Load Balancing*),

cachen häufig angeforderte Ressourcen, komprimieren Antworten und schützen die eigentlichen Anwendungsserver, die dadurch gar nicht erst direkt aus dem Internet erreichbar sein müssen. Ein typisches Beispiel: Eine Node.js-Anwendung hört auf Port 3000, weil Ports unterhalb 1024 unter Linux nur mit Root-Rechten geöffnet werden dürfen und eine Anwendung aus Sicherheitsgründen nicht als Root laufen soll. Browser erwarten HTTPS-Verbindungen aber auf Port 443. Die Lösung: Ein vorgelagerter nginx-Webserver als Reverse Proxy nimmt eingehende Verbindungen auf Port 443 entgegen, terminiert TLS und leitet die entschlüsselten Requests intern an die Node-Anwendung auf Port 3000 weiter. Für den Client sieht alles wie eine ganz normale HTTPS-Verbindung aus, die interne Port-Verschiebung bleibt unsichtbar. Dasselbe Architektur-Muster findet sich auch bei PHP-Anwendungen in Produktion wieder: Dort trennt man die Auslieferung statischer Dateien (HTML, CSS, JavaScript, Bilder) von der Ausführung des PHP-Codes. Ein schlanker Webserver wie nginx oder Apache im event-basierten Modus nimmt alle Anfragen entgegen, liefert statische Ressourcen selbst aus und reicht PHP-Requests per FastCGI-Protokoll an einen separaten PHP-FPM-Prozess-Pool weiter, der den eigentlichen PHP-Code ausführt. Diese Trennung skaliert deutlich besser als ältere Ansätze, bei denen ein vollständiger PHP-Interpreter in jeden Webserver-Worker eingebettet war – schon weil sich Worker für statische und dynamische Inhalte unabhängig dimensionieren lassen.

Noch eine Stufe weiter außen steht oft eine *Firewall*. Sie filtert den ein- und ausgehenden Netzwerkverkehr anhand von Regeln – typischerweise auf Basis von Quell- und Ziel-IP-Adressen sowie Ports. Für einen Webserver ist eine gängige Regel etwa: „Erlaube eingehende Verbindungen auf Port 443, blockiere alles andere.“ Firewalls sind für unsere Betrachtung der Kommunikationswege primär ein zusätzlicher Kontrollpunkt, den ein Paket auf seinem Weg passieren muss; sie ändern am eigentlichen HTTP-Ablauf nichts.

1.5 Schichtenmodelle: OSI und TCP/IP

Die Kommunikation über ein Netzwerk ist komplex. Damit sie überhaupt handhabbar wird, wird sie gedanklich in mehrere *Schichten* zerlegt. Jede Schicht ist für einen klar abgegrenzten Aufgabenbereich zuständig und bietet der jeweils darüberliegenden Schicht eine definierte Schnittstelle an. Die unteren Schichten kümmern sich um physikalische und lokale Aspekte (wie werden Bits über ein Kabel übertragen?), die oberen Schichten um anwendungsbezogene Dinge (wie sieht ein HTTP-Request inhaltlich aus?).

1.5.1 Das OSI-Referenzmodell

Der bekannteste theoretische Entwurf ist das OSI-Referenzmodell (Open Systems Interconnection), das 1984 erstmals von der ISO veröffentlicht und 1994 in seiner heute gültigen Form als ISO/IEC,7498-1 standardisiert wurde [6].

1. **Bitübertragungsschicht** (Physical Layer): legt fest, wie Nullen und Einsen elektrisch, optisch oder per Funk auf das Übertragungsmedium (Kupferkabel, Glasfaser, Luft) aufgebracht werden.
2. **Sicherungsschicht** (Data Link Layer): kümmert sich um die physikalische Adressierung (MAC-Adressen) und stellt mittels Prüfsummen sicher, dass die Daten korrekt übertragen wurden. Formal teilt sie sich in zwei Unterschichten: *LLC* (Logical

Tabelle 1.2: Vergleich von OSI- und TCP/IP-Modell. Für das Szenario eines Webrequests sind vor allem die drei mittleren Ebenen relevant: HTTP auf der Anwendungsschicht, TCP auf der Transportschicht und IP auf der Vermittlungsschicht.

OSI	TCP/IP	Protokolle (Auswahl)	Dateneinheit
7 Anwendung	Anwendung	HTTP(S), SMTP, DNS, SSH	Nachricht / Daten
6 Darstellung			
5 Sitzung			
4 Transport	Transport	TCP, UDP	Segment / Datagramm
3 Vermittlung	Internet	IP, ICMP	Paket
2 Sicherung	Netzzugang	Ethernet, WLAN, ARP	Frame
1 Bitübertragung		Kupfer, Glasfaser, Funk	Bit

Link Control) für Fluss- und Fehlerkontrolle und *MAC* (Media Access Control) für die Hardware-Adressierung und den Zugriff auf das Übertragungsmedium.

3. **Vermittlungsschicht** (Network Layer): logische Adressierung, Routenfindung (Routing), Datenflusskontrolle zwischen Netzen.
4. **Transportschicht** (Transport Layer): ordnet Daten einer bestimmten Anwendung auf dem Zielrechner zu und kümmert sich um Zuverlässigkeit und Reihenfolge der Übertragung.
5. **Sitzungsschicht** (Session Layer): Auf- und Abbau sowie Verwaltung von Sitzungen auf Anwendungsebene.
6. **Darstellungsschicht** (Presentation Layer): Umwandlung zwischen systemabhängigen und systemunabhängigen Datenformaten, Zeichenkodierung.
7. **Anwendungsschicht** (Application Layer): Funktionen für konkrete Anwendungen wie Webbrowser, E-Mail-Programme oder Dateitransfer-Tools.

1.5.2 Das TCP/IP-Modell in der Praxis

In der Praxis hat sich ein etwas schlankeres Modell durchgesetzt, das *TCP/IP-Modell*. Es fasst einige der OSI-Schichten zusammen und ist näher an den tatsächlich eingesetzten Protokollen. Tabelle 1.2 stellt die beiden Modelle samt typischer Protokolle gegenüber.

Das OSI-Modell ist ein sauberer theoretischer Entwurf, das TCP/IP-Modell ist jedoch aus der realen Entwicklung des Internets heraus entstanden und deshalb näher an dem, was tatsächlich auf der Leitung liegt. In der Fehlerdiagnose denkt man jedoch meist in OSI-Begriffen: Wenn ein entfernter Rechner nicht erreichbar ist, sich aber anpingen lässt, ist das Problem vermutlich auf der Anwendungsschicht; lässt er sich nicht einmal pinggen, liegt das Problem eine oder mehrere Schichten darunter.

1.5.3 Encapsulation: Wie Schichten zusammenspielen

Die Schichten arbeiten zusammen, indem jede von ihnen die Daten der darüberliegenden Schicht als *Nutzdaten* (*Payload*) behandelt und einen eigenen *Header* davorsetzt. Dieses

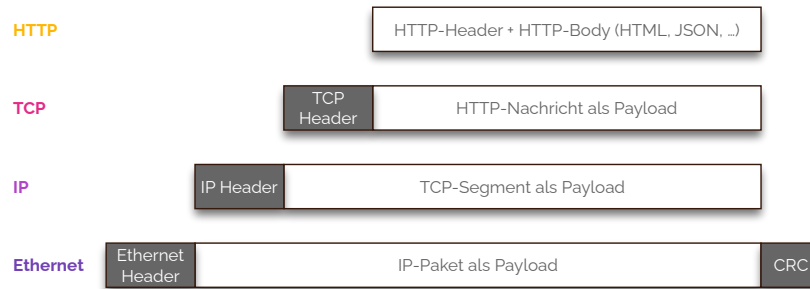


Abbildung 1.4: Encapsulation am Beispiel eines HTTP-Requests: Jede Schicht erweitert die Daten der darüberliegenden Schicht um einen eigenen Header. Ethernet fügt zusätzlich einen Trailer mit einer Prüfsumme (CRC) an, über die sich Übertragungsfehler erkennen lassen.

Prinzip heißt *Encapsulation*. Es bedeutet konkret: Wenn der Browser einen HTTP-Request losschickt, wird dieser Request zunächst als HTTP-Nachricht formuliert, dann mit einem TCP-Header versehen, dann mit einem IP-Header, dann mit einem Ethernet-Header (und einem Ethernet-Trailer mit Prüfsumme). Abbildung 1.4 zeigt das Prinzip.

Am Zielrechner läuft der Prozess in umgekehrter Richtung ab: Der Ethernet-Header wird entfernt und auf Basis seiner Informationen entschieden, dass die Nutzdaten an die IP-Schicht weitergereicht werden. Diese entfernt den IP-Header und übergibt an die TCP-Schicht, die wiederum ihren Header entfernt und die HTTP-Nachricht an den Webserver-Prozess weiterreicht. So weiß jede Schicht ausschließlich, wie ihr eigener Header aussieht und welche Schicht darüberliegt – alles andere ist für sie undurchsichtiger Payload.

Eine konkrete Kombination aufeinanderfolgender Protokolle nennt man *Protokoll-stack*. Für das Szenario eines Webrequests ist der relevante Stack also HTTP → TCP → IP → Ethernet (oder WLAN), für eine DNS-Anfrage (siehe Abschnitt 1.6) wäre es DNS → UDP → IP → Ethernet.

1.6 Vom Namen zur Adresse: DNS

Zurück zum Webrequest-Beispiel: Der Browser hat den URL zerlegt und weiß, dass er mit einem Host namens `www.fh-ooe.at` sprechen will. Rechner im Internet werden aber nicht über Namen adressiert, sondern über *IP-Adressen* – also über Nummern wie 162.55.28.238 (IPv4) oder 2001:62a:4:417::5 (IPv6). Der Name muss also zuerst in eine IP-Adresse übersetzt werden. Diese Aufgabe erledigt das *Domain Name System* (DNS).

1.6.1 Hostnamen, Domains und FQDN

Ein Hostname wie `www.fh-ooe.at` ist hierarchisch aufgebaut und wird *von rechts nach links* gelesen:

- `at` ist die *Top-Level-Domain* (TLD). Für sie ist eine eigene Verwaltungsorganisation zuständig (im Fall von `.at` die nic.at).

- **fh-ooe** ist die *Second-Level-Domain*. Sie wurde bei der zuständigen Organisation registriert und gehört einem bestimmten Inhaber (hier der FH Oberösterreich).
- **www** ist der *Hostname* innerhalb dieser Domain. Er bezeichnet einen konkreten Rechner (oder Dienst) innerhalb der Domain.

Die komplette Kette bis zur TLD nennt man *Fully Qualified Domain Name* (FQDN).

1.6.2 Ablauf der Namensauflösung

Der Browser selbst führt die Namensauflösung nicht direkt durch. Stattdessen fragt das Betriebssystem einen *DNS-Resolver*, der in der Regel beim Internet-Provider steht und dessen IP-Adresse der Rechner üblicherweise per DHCP mitgeteilt bekommen hat (mehr dazu in Abschnitt 1.7.4). Dieser Resolver fragt dann bei Bedarf rekursiv weitere DNS-Server an, bis er eine Antwort hat:

1. Zunächst wird ein *Root-Nameserver* befragt – einer von 13 weltweit verteilten Servern, die den obersten Einstiegspunkt ins DNS bilden und deren Adressen in jeder Resolver-Software fest eingebaut sind. Die Frage lautet: „Wer ist für **.at** zuständig?“
2. Die Antwort verweist auf einen Nameserver der **.at**-Zone. Dieser wird gefragt: „Wer ist für **fh-ooe.at** zuständig?“
3. Die Antwort verweist auf den autoritativen Nameserver der FH Oberösterreich. Dieser wird schließlich gefragt: „Wie lautet die IP-Adresse von **www.fh-ooe.at**?“
4. Die liefert die eigentliche IP-Adresse zurück.

In der Praxis ist dieser Prozess durch aggressives *Caching* auf mehreren Ebenen stark beschleunigt. Der Resolver speichert bereits aufgelöste Namen für eine bestimmte Zeit (die *Time-to-Live*, TTL), und auch das lokale Betriebssystem und der Browser haben eigene DNS-Caches. Für eine populäre Seite wie **www.fh-ooe.at** fällt in den allermeisten Fällen gar keine externe Anfrage an – die Antwort kommt direkt aus dem Cache des Resolvers.

DNS verwendet überwiegend *UDP* als Transportprotokoll (Port 53), weil einzelne kurze Anfragen ohne Verbindungsaufbau effizienter sind als mit TCP. Das passt zu der Faustregel: UDP dort, wo kleine Einzeldatagramme und niedrige Latenz gefragt sind; TCP dort, wo eine verlässliche, geordnete Datenübertragung über längere Zeit notwendig ist (siehe dazu Abschnitt 1.8).

Am Ende dieses Schritts kennt der Browser die IP-Adresse des Webserver, etwa **162.55.28.238**. Offen bleibt nun die Frage, *wie* die Anfrage tatsächlich dorthin gelangt. Abbildung 1.5 stellt den Ablauf dar.

1.7 Die Netzwerkschicht: IP und lokale Zustellung

Mit der aufgelösten IP-Adresse des Webserver kann die eigentliche Zustellung beginnen. Die dafür zuständige Schicht ist die Netzwerkschicht, auf der das *Internet Protocol* (IP) arbeitet. Die folgenden Unterabschnitte behandeln die wichtigsten Bausteine, die für diese Zustellung erforderlich sind: die IP-Adressierung selbst, die automatische Netzwerkkonfiguration per DHCP, die Übersetzung von IP- auf Hardware-Adressen per

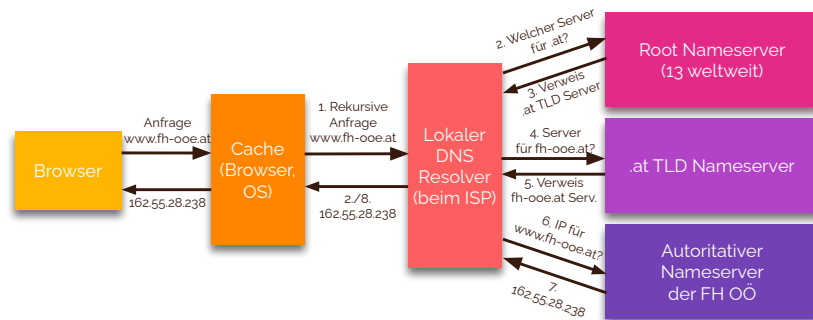


Abbildung 1.5: Der Ablauf vom Namen **www.fh-ooe.at** bis zur IP-Adresse, inklusive möglichem Caching.

ARP, den Aufbau von Ethernet-Frames und schließlich die Netzwerkkomponenten, die die Pakete tatsächlich weiterleiten.

1.7.1 IP-Adressen

IP-Adressen sind die logischen Adressen, unter denen Rechner im Internet erreichbar sind. *IPv4*-Adressen sind 32 ($4 \cdot 8$) Bit lang und werden in der gewohnten Punktschreibweise notiert (162.55.28.238). Da der IPv4-Adressraum viel zu klein für das heutige Internet ist, gibt es parallel den Nachfolger *IPv6* mit 128 Bit langen Adressen, die in Hexadezimal-Blöcken notiert werden (2a01:04f8:2190:2103:d64a:3471:c57f:8d4c). Für unsere Betrachtung ist der Unterschied unerheblich.

Jede IP-Adresse zerfällt in zwei Teile: den *Netzanteil* (links) und den *Hostanteil* (rechts). Der Netzanteil identifiziert das Netzwerk, in dem sich der Rechner befindet; der Hostanteil identifiziert den konkreten Rechner innerhalb dieses Netzes. Wie lang der Netzanteil ist, bestimmt die *Subnetzmaske*. Für ein Netz, in dem die ersten 24 Bit zum Netzanteil gehören, lautet die Subnetzmaske dezimal 255.255.255.0. Statt der ausführlichen Maske hat sich heute die kürzere *CIDR-Schreibweise* (Classless Inter-Domain Routing) durchgesetzt: Man hängt an die Netzadresse einen Schrägstrich und die Anzahl der Netzanteil-Bits an, also beispielsweise 192.168.0.0/24. Diese Notation begegnet in der Praxis ständig – in Router-Konfigurationen, in Cloud-Dashboards und im Output von Kommandos wie `ip route`.

Ein paar konkrete Beispiele machen den Zusammenhang anschaulich:

Adresse	Subnetzmaske	CIDR	Bedeutung
192.168.0.17	255.255.255.0	/24	Host im kleinen Heimnetz
10.42.7.200	255.255.0.0	/16	Host in großem Firmennetz
172.16.5.3	255.255.255.240	/28	Host in kleinem Subnetz

Im ersten Beispiel gehören die ersten drei Oktette (192.168.0) zum Netzanteil, das letzte Oktett (17) ist der Hostanteil. Im /24-Netz sind dadurch 254 Host-Adressen nutzbar. Zunächst wird von den gesamten 32 Bit einer IPv4-Adresse der Hostanteil abgezogen, also $32 - 24 = 8$, es bleiben also 8 Bit für die Hosts, $2^8 = 256$, somit wären theoretisch 256 Hostadressen möglich. Für die nutzbaren Hosts müssen jedoch immer

2 abgezogen werden: die erste Adresse (hier 192.168.0.0), genannt *Netz-ID* oder *Netz-Adresse* und die letzte Adresse (hier 192.168.0.255), genannt *Broadcast-Adresse*. Somit verbleiben 254 nutzbare Hosts, von 192.168.0.1 bis 192.168.0.254.

Im zweiten Beispiel umfasst der Netzanteil nur die ersten beiden Oktette (10.42), und es bleiben 16 Bit für Hosts – also $65.534 (2^{16} - 2)$ mögliche Adressen.

Das dritte Beispiel zeigt ein bewusst kleines Subnetz: Bei /28 werden 28 Bit für das Netz verwendet und nur 4 Bit bleiben für Hosts übrig, was lediglich 14 nutzbare Adressen ($2^4 - 2$) ergibt.

Rechner, die im selben Subnetz sind, können sich direkt auf Ethernet-Ebene miteinander unterhalten. Rechner in unterschiedlichen Subnetzen müssen über einen *Router* kommunizieren. Der Router des eigenen Subnetzes wird als *Standard-Gateway* (*Default Gateway*) bezeichnet. Ob ein Ziel im gleichen Subnetz liegt oder nicht, lässt sich anhand der Subnetzmaske durch einen einfachen Bit-Vergleich ermitteln – Details dazu sind aber vertiefender Stoff der Netzwerktechnik und werden hier nicht weiter thematisiert.

1.7.2 Unicast, Broadcast und Multicast

Pakete auf IP-Ebene werden in drei unterschiedlichen Modi verschickt:

Unicast Ein Paket hat genau einen Absender und genau einen Empfänger. Das ist der Normalfall und betrifft praktisch die gesamte Web-Kommunikation: Ein HTTP-Request geht von genau einem Browser an genau einen Webserver.

Broadcast Ein Paket geht an *alle* Rechner im lokalen Subnetz. Dafür gibt es eine spezielle Broadcast-Adresse (in einem 192.168.0.0/24-Netz etwa 192.168.0.255). Broadcast wird unter anderem von DHCP und ARP verwendet, weil ein neu hinzukommender Rechner zunächst noch gar nicht weiß, wen er gezielt ansprechen könnte.

Multicast Ein Paket geht an eine bestimmte *Gruppe* von Rechnern, die sich zuvor für einen gemeinsamen Kanal angemeldet haben. Für diese Betrachtung spielt Multicast keine Rolle, er wird vor allem für Medienstreams in lokalen Netzen eingesetzt.

Broadcasts werden von Routern bewusst *nicht* weitergeleitet – sonst würde ein einzelner DHCP-Broadcast das gesamte Internet überschwemmen. Sie bleiben immer auf das lokale Subnetz beschränkt.

1.7.3 Private Adressen und NAT

An dieser Stelle stellt sich die Frage, warum der Laptop im Beispiel eine Adresse wie 192.168.0.17 erhält – solche Adressen können ja nicht beliebig „im Internet“ vergeben werden, ohne sich mit anderen Nutzer*innen zu überschneiden. Die Antwort: Bestimmte IPv4-Bereiche sind explizit als *private Adressen* reserviert und dürfen innerhalb beliebiger lokaler Netze verwendet werden. Die drei Bereiche sind:

CIDR	Adressbereich
10.0.0.0/8	10.0.0.0–10.255.255.255
172.16.0.0/12	172.16.0.0–172.31.255.255
192.168.0.0/16	192.168.0.0–192.168.255.255

Pakete mit solchen Quell- oder Zieladressen werden im öffentlichen Internet nicht geroutet. Damit ein Rechner aus einem privaten Netz trotzdem Webseiten abrufen kann, kommt *Network Address Translation* (NAT) zum Einsatz: Der Router am Übergang zwischen privatem Netz und Internet ersetzt in ausgehenden Paketen die private Quell-IP durch seine eigene öffentliche IP (und merkt sich die Zuordnung anhand der Portnummern). Kommen Antwortpakete zurück, macht er die Übersetzung rückgängig und leitet sie an den richtigen internen Rechner weiter. Aus Sicht des Webservers kommuniziert er dabei immer nur mit dem NAT-Router; die dahinterliegenden Rechner sind für ihn nicht sichtbar. NAT ist ein zentraler Grund dafür, dass IPv4 trotz seines viel zu kleinen Adressraums bis heute funktioniert.

Für die serverseitige Entwicklung ist NAT unter anderem dann relevant, wenn PHP-Skripte die IP-Adresse des aufrufenden Clients auswerten wollen (z. B. für Geolocation oder Logging): Die „gesehene“ IP-Adresse ist dann in vielen Fällen nicht die tatsächliche IP des Clients, sondern die eines zwischengeschalteten NAT-Routers oder Reverse Proxys (vgl. Abschnitt 1.4.1).

1.7.4 DHCP: Automatische Netzwerkkonfiguration

Woher bezieht der Laptop im Beispiel eigentlich seine eigene IP-Adresse? In modernen Netzen wird diese dynamisch vergeben. Verantwortlich dafür ist das *Dynamic Host Configuration Protocol* (DHCP). Sobald sich der Laptop mit dem WLAN verbindet, schickt er einen DHCP-Request als *Broadcast* ins lokale Netz. Ein DHCP-Server (meist der Router selbst) antwortet mit einem Datenpaket, das mindestens folgende Informationen enthält:

- eine freie IP-Adresse für den Laptop,
- die zugehörige Subnetzmaske,
- die IP-Adresse des Standard-Gateways und
- die IP-Adresse(n) eines oder mehrerer DNS-Resolver.

Erst nach erfolgreichem DHCP-Vorgang kennt der Laptop also seine eigene IP-Adresse, weiß, wohin er Pakete für entfernte Rechner schicken soll (zum Gateway), und kann DNS-Anfragen stellen. Unter Windows lassen sich diese Informationen bequem mit dem Kommando `ipconfig /all` anzeigen, unter Linux/macOS mit `ifconfig` bzw. `ip addr` und `ip route`.

1.7.5 ARP: Vom IP zum MAC

Der Laptop weiß jetzt Folgendes: Seine eigene IP ist z. B. 192.168.0.17, die des Gateways 192.168.0.1, und der Webserver liegt unter 162.55.28.238. Der Webserver ist offensichtlich nicht im lokalen Subnetz (anderer Adressbereich), also müssen die Pakete an das Gateway geschickt werden. Aber Ethernet kennt keine IP-Adressen – es kennt nur *MAC-Adressen* (Media Access Control). Eine MAC-Adresse ist 48 Bit lang und weltweit (fast) eindeutig, bestehend aus einer Herstellerkennung und einer fortlaufenden Nummer, beispielsweise 00:0e:7f:27:a8:59.

Um die Ethernet-Frames korrekt adressieren zu können, braucht der Laptop also die MAC-Adresse des Gateways. Diese ermittelt er mit dem *Address Resolution Protocol* (ARP): Er schickt einen ARP-Request als Broadcast ins lokale Netz („Wer hat die

IP-Adresse 192.168.0.1?“), und das Gateway antwortet mit einer unicast-Nachricht zurück („Das bin ich, meine MAC-Adresse ist 00:0e:7f:27:a8:59“). Diese Zuordnung wird in einem lokalen ARP-Cache zwischengespeichert, damit sie nicht für jedes einzelne Paket neu ermittelt werden muss.

1.7.6 Ethernet-Frames und lokale Zustellung

Ethernet ist das mit Abstand am häufigsten eingesetzte Protokoll der Sicherungsschicht (OSI Schicht 2) und regelt die Datenübertragung auf dem lokalen Netzwerksegment, also etwa zwischen einem Laptop und dem Switch oder Router im selben Raum. Die Dateneinheit auf dieser Schicht heißt *Frame*: Ein Ethernet-Frame umschließt das eigentliche IP-Paket mit einem Header (Quell- und Ziel-MAC-Adresse sowie ein Typfeld, das angibt, welches Protokoll der nächsthöheren Schicht eingebettet ist) und einem Trailer mit einer Prüfsumme. Die maximale Größe der Nutzdaten in einem Standard-Frame liegt bei 1500 Byte – dieser Wert wird im TCP-Abschnitt 1.8 noch eine Rolle spielen.

Mit diesem Wissen kann der Laptop nun das erste Frame zusammenstellen. Die Netzwerkkarte schickt es auf das Medium; der Netzwerk-Switch, an dem der Laptop hängt, liest die Ziel-MAC aus, stellt fest, hinter welchem seiner Ports sich der entsprechende Rechner befindet, und leitet das Frame gezielt nur dorthin weiter – in unserem Fall an das Gateway. Alle anderen angeschlossenen Rechner bekommen das Frame gar nicht zu sehen.

Das Gateway schaut sich nun den IP-Header an, sieht die Ziel-IP 162.55.28.238, stellt fest, dass es das nicht selbst ist, und leitet das Paket entsprechend seiner Routing-Tabelle weiter – bei jedem Hop (Router) wird dabei nur der Ethernet-Header getauscht; der IP-Header, der TCP-Header und die HTTP-Nutzdaten bleiben unverändert. So bewegt sich das Paket schließlich über mehrere Router durch das Internet bis zum Webserver.

1.7.7 Netzwerkkomponenten: Hub, Switch, Router

An dieser Stelle lohnt es sich, die Geräte, die auf dem Weg eines Pakets begegnen, nach ihrer Funktion zu sortieren. Die gängige Einteilung erfolgt nach der OSI-Schicht, auf der sie arbeiten:

Hub (Schicht 1) Ein Hub ist im Kern ein dummer Signalverstärker: Was auf einem Port hereinkommt, wird einfach auf alle anderen Ports kopiert. Alle angeschlossenen Rechner sehen den gesamten Datenverkehr, was zu Kollisionen und geringer Effizienz führt. In der Praxis sind Hubs heute praktisch ausgestorben und wurden fast überall durch Switches ersetzt.

Switch (Schicht 2) Ein Switch lernt mit, welche MAC-Adressen hinter welchem seiner Ports liegen, und leitet jedes Frame gezielt nur an den Port weiter, an dem die Ziel-MAC hängt. Dadurch kann jeder Port die volle Bandbreite nutzen, und Rechner sehen nur den Datenverkehr, der sie tatsächlich betrifft. Switches arbeiten mit MAC-Adressen und wissen von IP gar nichts.

Router (Schicht 3) Ein Router verbindet verschiedene Netze miteinander und trifft seine Weiterleitungsentscheidungen auf Basis von IP-Adressen. Er führt eine *Routing-Tabelle*, in der er nachschlägt, über welches seiner Interfaces ein Paket mit

einer gegebenen Ziel-IP weiterverschickt werden muss. Router sind die Stellen im Netz, an denen Pakete das lokale Subnetz verlassen.

In unserem Szenario passiert das Ethernet-Frame des Laptops also zuerst einen Switch im lokalen Netz (der es zum Gateway zustellt), und ab dann eine ganze Reihe von Routern auf dem Weg durchs Internet. Bei jedem Router-Hop wird der Ethernet-Header verworfen und ein neuer für das nächste Teilstück erzeugt, während der IP-Header unverändert bleibt.

1.7.8 Diagnose: ICMP, ping und traceroute

Eng mit IP verwandt ist das *Internet Control Message Protocol* (ICMP). ICMP ist auf derselben Schicht angesiedelt wie IP und dient dazu, Status und Fehlermeldungen im Netz zu transportieren – etwa „Zielrechner nicht erreichbar“ oder „Time-to-Live abgelaufen“. Im Kontext der Webentwicklung ist ICMP vor allem deshalb relevant, weil zwei häufig eingesetzte Diagnose-Tools darauf aufsetzen:

ping schickt ein *Echo Request* an eine angegebene IP-Adresse oder einen Hostnamen. Antwortet die Gegenseite mit einem *Echo Reply*, ist der Zielrechner grundsätzlich erreichbar und es besteht eine funktionierende IP-Verbindung dorthin. Bleibt die Antwort aus, liegt das Problem auf der Netzwerk- oder Vermittlungsschicht – HTTP oder die Webanwendung sind dann gar nicht erst betroffen. Typischer Aufruf: `ping www.fh-ooe.at`

traceroute (unter Windows `tracert`) zeigt den Weg, den ein Paket bis zu seinem Ziel nimmt, und listet alle Zwischen-Router auf. Dabei wird die TTL (*Time to Live*) im IP-Header gezielt manipuliert: Der jeweils erste Router, bei dem sie auf null fällt, meldet dies per ICMP zurück und gibt sich damit zu erkennen. So lässt sich feststellen, an welcher Stelle auf dem Weg ein Paket „hängenbleibt“.

Beide Werkzeuge sind für die Diagnose serverseitiger Probleme unentbehrlich und dienen typischerweise als erster Prüfschritt, wenn eine Webanwendung nicht mehr erreichbar ist. Reagiert der Server auf **ping**, während HTTPS-Verbindungen fehlschlagen, so ist zumindest gesichert, dass die Netzwerkverbindung steht und das Problem auf einer höheren Ebene liegt – etwa beim Webserver, bei TLS oder bei der Anwendung selbst.

Einschränkend sei angemerkt: Aus Sicherheitsgründen blockieren manche Server und Firewalls eingehende ICMP-Pakete. Ein fehlgeschlagener **ping** ist deshalb nicht in jedem Fall ein zwingendes Indiz dafür, dass der Server tatsächlich nicht erreichbar ist.

1.8 Die Transportschicht: TCP

Auf der Netzwerkschicht ist IP ein *verbindungsloses und unzuverlässiges* Protokoll. Das klingt schlimmer, als es ist: IP liefert Pakete so schnell und so gut es geht zu, garantiert aber weder, dass sie ankommen, noch, dass sie in der richtigen Reihenfolge ankommen, noch dass sie nicht dupliziert werden. Für einen Webrequest soll aber garantiert sein, dass *alle* Teile der Anfrage und der Antwort vollständig und in der richtigen Reihenfolge bei dem*der Empfänger*in ankommen. Darum kümmert sich die Transportschicht, konkret das *Transmission Control Protocol* (TCP).

1.8.1 TCP und UDP

Neben TCP gibt es noch das *User Datagram Protocol* (UDP). Die beiden Protokolle ergänzen sich und werden für unterschiedliche Anwendungsfälle eingesetzt. Eine Gegenüberstellung macht auch die Funktionsweise deutlich:

TCP baut eine Verbindung zwischen zwei Endpunkten auf, sorgt für die zuverlässige Übertragung und die korrekte Reihenfolge, fordert verlorene Pakete erneut an und steuert die Datenrate. TCP wird von HTTP(S), SSH, SMTP, FTP und vielen anderen Anwendungen verwendet.

UDP verschickt einzelne Datagramme ohne Verbindungsaufbau und ohne Zustellgarantie. Es ist deutlich schlanker und eignet sich für kurze Anfragen (DNS), für Echtzeitanwendungen wie VoIP und Video-Streaming, bei denen ein paar verlorene Pakete verschmerzbar sind, oder für Broadcasts (DHCP).

Gemeinsam ist beiden, dass sie über *Port-Nummern* adressieren, welcher Prozess auf dem Zielrechner die Daten bekommen soll. Ein Port ist eine 16-Bit-Zahl (also zwischen 0 und 65535). Für bekannte Dienste gibt es fest vergebene *Well-Known Ports*:

Port	Dienst
20, 21	FTP
22	SSH
25	SMTP
53	DNS
67, 68	DHCP
80	HTTP
443	HTTPS

Ports unterhalb 1024 sind reserviert und werden typischerweise nur von Serverdiensten verwendet; Clients wählen sich beim Verbindungsaufbau einen freien Port oberhalb von 1024 aus (*ephemeral port*).

1.8.2 Der Three-Way-Handshake

Bevor Daten über eine TCP-Verbindung fließen können, muss diese aufgebaut werden. Das geschieht mit einem sogenannten *Three-Way-Handshake*, dargestellt in Abbildung 1.6:

1. Der Client (Browser) schickt ein TCP-Segment mit gesetztem SYN-Flag an den Server, Zielport 443. Damit signalisiert er „Ich möchte eine Verbindung öffnen“ und teilt gleichzeitig seine Anfangs-Sequenznummer mit.
2. Der Server antwortet mit einem Segment, das sowohl SYN als auch ACK gesetzt hat: Er bestätigt die Sequenznummer des Clients und sendet seine eigene.
3. Der Client bestätigt seinerseits mit einem reinen ACK.

Ab diesem Moment steht eine bidirektionale TCP-Verbindung. Jede Seite kann nun Daten schicken, die automatisch in der richtigen Reihenfolge wieder zusammengesetzt werden. Fällt ein Paket aus, wird es erneut angefordert; ist die Empfängerseite überlastet, wird die Senderate gedrosselt (*Flow Control*). All das läuft transparent ab – die Anwendungsschicht (also HTTP) bekommt davon nichts mit.

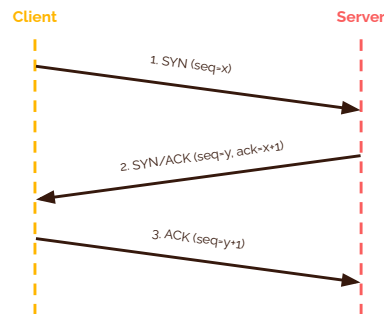


Abbildung 1.6: TCP-Verbindungsaufbau mit dem Three-Way-Handshake. Der Client initiiert die Verbindung mit einem SYN-Paket, der Server bestätigt (SYN/ACK) und der Client quittiert seinerseits (ACK). Erst nach diesen drei Paketen steht die Verbindung.

Damit das Reihenfolge-Versprechen eingelöst werden kann, muss TCP größere Datenmengen zunächst in passende Häppchen zerlegen. Der Grund dafür liegt eine Schicht tiefer: Ein Ethernet-Frame (siehe Abschnitt 1.7.6) kann in der Standardkonfiguration nur bis zu 1.500 Byte Nutzdaten tragen (die *Maximum Transmission Unit*, MTU). Größere HTTP-Nachrichten – etwa ein mehrere Megabyte großes Bild oder ein längeres HTML-Dokument – werden deshalb in mehrere TCP-Segmente unterteilt. Jedes Segment bekommt im TCP-Header eine *Sequenznummer*, die angibt, an welcher Position im Gesamtdatenstrom seine Nutzdaten stehen. Am Empfänger werden die Segmente anhand dieser Sequenznummern wieder in die richtige Reihenfolge gebracht, auch wenn sie unterwegs unterschiedliche Routen genommen haben und in einer anderen Reihenfolge eingetroffen sind. Fehlende Sequenznummern verraten verlorene Pakete und lösen die erneute Übertragung aus.

Der Verbindungsabbau erfolgt analog mit FIN-Paketen, weitere Details können wieder entsprechender Netzwerktechnik-Literatur entnommen werden.

1.9 Sichere Übertragung: HTTPS und TLS

Bisher wurde implizit angenommen, dass die Kommunikation zwischen Browser und Webserver offen erfolgt. Für viele Anwendungen ist das jedoch nicht akzeptabel: Passwörter, Zahlungsdaten, persönliche Informationen oder auch nur der konkret aufgerufene Inhalt sollen nicht für jede Person abhörbar sein, die irgendwo entlang der Leitung Zugriff auf den Netzwerkverkehr erlangen könnte. Deshalb verwendet der Beispiel-URL zur Seite der FH Oberösterreich nicht `http://`, sondern `https://`.

1.9.1 HTTPS ist HTTP über TLS

HTTPS ist kein eigenständiges Protokoll, sondern einfach HTTP, das durch eine zusätzliche Schicht namens *TLS* (Transport Layer Security, früher SSL) geschützt wird. TLS schiebt sich zwischen TCP und HTTP ein und sorgt für drei Dinge:

- **Vertraulichkeit** (*confidentiality*) – die übertragenen Daten werden verschlüsselt, sodass Dritte sie nicht mitlesen können.

- **Integrität** (*integrity*) – Änderungen an den Daten während der Übertragung werden bemerkt.
- **Authentizität** (*authenticity*) – der Client kann sich sicher sein, tatsächlich mit dem Server zu sprechen, dessen Name im URL steht, und nicht mit einem*iner Angreifer*in, der*die sich dazwischengeschaltet hat.

Im Schichtenmodell befindet sich TLS oberhalb der Transportschicht und unterhalb der Anwendungsprotokolle. Im OSI-Modell wird es üblicherweise der *Sitzungsschicht* (Schicht 5) zugeordnet. Im schlankeren TCP/IP-Modell gibt es keine eigene Ebene dafür, und TLS wird dort als transparente Zwischenschicht zwischen Transport- und Anwendungsschicht eingeordnet.

Aus Sicht der HTTP-Nachricht ist TLS transparent: Die HTTP-Nachricht wird vor dem Übergeben an TCP verschlüsselt und bei dem*der Empfänger*in wieder entschlüsselt. Der HTTP-Request selbst sieht genauso aus wie im unverschlüsselten Fall.

1.9.2 Der TLS-Handshake

Bevor Nutzdaten verschlüsselt übertragen werden können, muss zwischen Client und Server ein gemeinsamer Schlüssel ausgehandelt werden, und der Client muss überprüfen können, dass der Server tatsächlich der ist, für den er sich ausgibt. Das geschieht im *TLS-Handshake*, der gleich nach dem TCP-Handshake stattfindet. Stark vereinfacht läuft er so ab:

1. Der Client sendet ein **ClientHello**, in dem er seine unterstützten TLS-Versionen, Cipher Suites (d. h. Verschlüsselungsverfahren) und eine Zufallszahl mitteilt.
2. Der Server antwortet mit einem **ServerHello**, in dem er die Auswahl der TLS-Version und der Cipher Suite trifft und ebenfalls eine Zufallszahl schickt. Zusätzlich übermittelt er sein *Zertifikat*.
3. Der Client überprüft das Zertifikat: Ist es gültig? Wurde es von einer ihm bekannten Zertifizierungsstelle ausgestellt? Passt der im Zertifikat angegebene Name zum Host, mit dem gerade gesprochen wird? Schlägt auch nur eine dieser Prüfungen fehl, bricht der Browser die Verbindung ab und zeigt dem*der Anwender*in eine Warnung an.
4. Anschließend führen beide Seiten durch asymmetrische Kryptografie einen Schlüsselaustausch durch und leiten daraus einen gemeinsamen *symmetrischen* Sitzungsschlüssel ab. Die eigentliche Datenübertragung erfolgt später symmetrisch, was wesentlich schneller ist.
5. Beide Seiten signalisieren mit einer **Finished**-Nachricht, dass ab jetzt verschlüsselt kommuniziert wird.

Das Ergebnis ist ein verschlüsselter Tunnel durch die bestehende TCP-Verbindung, in dem HTTP-Nachrichten vertraulich, unverändert und authentisch übertragen werden können.

1.9.3 Zertifikate und Zertifizierungsstellen

Ein *Zertifikat* im Sinne von TLS ist ein digital signiertes Dokument, das den öffentlichen Schlüssel eines Servers mit einem Domain-Namen verknüpft und von einer *Zer-*

tifizierungsstelle (Certification Authority, CA) ausgestellt wurde. Eine bekannte (und kostenlose) CA ist *Let's Encrypt*⁶, daneben gibt es kommerzielle Anbieter wie *DigiCert*, *Sectigo* oder *GlobalSign*.

Damit Browser Zertifikate überhaupt überprüfen können, liefern sie eine vorinstallierte Liste vertrauenswürdiger CAs mit (den sogenannten *Trust Store*). Wenn das Zertifikat eines Webservers von einer CA aus dieser Liste signiert wurde (oder von einer Zwischen-CA, die wiederum von einer vertrauenswürdigen CA signiert wurde), gilt es als vertrauenswürdig – der Browser zeigt das kleine Schloss-Symbol neben der URL an. Ansonsten erscheint eine Warnung.

Für Webentwickler*innen bedeutet das in der Praxis: Auf Produktivsystemen wird immer TLS eingesetzt, und der Webserver wird so konfiguriert, dass Anfragen auf Port 80 automatisch auf Port 443 umgeleitet werden. In der lokalen Entwicklungsumgebung (etwa mit *fhooe-web-dock*, das ab Vorlesung 2 zum Einsatz kommt) ist der TLS-Einsatz dagegen meist optional.

1.10 HTTP: Das Protokoll des Web

An diesem Punkt ist eine vollständige, verschlüsselte Verbindung zum Webserver aufgebaut. Über sie wird nun die eigentliche Anfrage verschickt. HTTP, das *Hypertext Transfer Protocol*, ist das Protokoll, über das im Web Inhalte ausgetauscht werden. Es ist ein *Anwendungsschicht-Protokoll* und folgt dem *Request-Response-Schema*: Der Client schickt einen Request, der Server antwortet mit einer Response, fertig. Der Server beginnt keine Kommunikation von selbst⁷.

1.10.1 Nachrichtenformat

HTTP-Nachrichten sind – zumindest in den Versionen 1.0 und 1.1 – *textbasiert* und damit bemerkenswert menschenlesbar. Eine HTTP-Nachricht besteht aus drei Teilen:

1. einer *Startzeile*,
2. einer Reihe von *Headerfeldern* (je ein Name-Wert-Paar pro Zeile) und
3. einem optionalen *Message Body*, der mit einer leeren Zeile vom Header-Block getrennt ist.

Ein einfacher HTTP-Request für das Beispielszenario sieht etwa so aus:

```
GET / HTTP/1.1
Host: www.fh-ooe.at
User-Agent: Mozilla/5.0 (...) Firefox/126.0
Accept: text/html,application/xhtml+xml,*/*;q=0.8
Accept-Language: de-AT,de;q=0.9,en;q=0.8
```

Die Startzeile besteht hier aus drei Teilen: der *Methode* (**GET**), dem *Request-URI* (**/**) und der *HTTP-Version* (**HTTP/1.1**). Danach folgen mehrere Headerfelder, dann eine

⁶<https://letsencrypt.org/>

⁷Dieser Grundsatz wird in neueren Versionen wie HTTP/2 durch sogenannte *Server-Pushes* etwas aufgeweicht, bleibt aber für unser Verständnis weiterhin eine gute Faustregel.

Leerzeile und im Fall eines GET-Requests kein Body.

Die Antwort des Servers könnte so aussehen:

```
HTTP/1.1 200 OK
Date: Mon, 09 Apr 2026 06:50:00 GMT
Server: Apache/2.4.58 (Ubuntu)
Content-Type: text/html; charset=UTF-8
Content-Length: 11321
```

```
<!DOCTYPE html>
<html lang="de">
...
</html>
```

Auch gibt es Startzeile, Headerblock, Leerzeile und Message Body. In der Startzeile einer Response stehen die HTTP-Version, ein *Statuscode* und eine zugehörige *Reason Phrase*.

1.10.2 HTTP-Methoden

Die *Methode* (auch *Verb*) gibt an, welche Art von Operation der Client auf der adressierten Ressource wünscht. Die wichtigsten Methoden sind:

GET Eine Ressource abrufen, ohne Seiteneffekte. GET-Anfragen dürfen vom Browser gecached und in die History aufgenommen werden. Parameter werden über den Query-String übergeben (siehe Abschnitt 1.3).

POST Daten an den Server senden, typischerweise zum Anlegen einer neuen Ressource oder zum Auslösen einer Aktion. Die Daten stehen im Message Body und sind daher in der URL nicht sichtbar. POST-Anfragen werden nicht gecached und nicht in der History gespeichert.

PUT Eine Ressource vollständig ersetzen oder anlegen.

DELETE Eine Ressource löschen.

PATCH Eine Ressource teilweise ändern.

HEAD Wie GET, aber der Server liefert nur die Header ohne Body. Nützlich, um z. B. die Größe einer Datei zu ermitteln, ohne sie komplett zu laden.

OPTIONS Fragt ab, welche Methoden auf einer bestimmten Ressource erlaubt sind. Spielt eine wichtige Rolle bei CORS (siehe Kapitel 11).

Für diese Lehrveranstaltung sind vor allem GET und POST wichtig. Kapitel 11 widmet sich den HTTP-Methoden im Detail.

1.10.3 Statuscodes

Der Server signalisiert in jeder Response mit einem dreistelligen *Statuscode*, wie es der Anfrage ergangen ist. Die erste Ziffer grenzt die Klasse ein:

1xx – *Informational* Zwischenmeldung, selten direkt sichtbar.

2xx – *Successful* Die Anfrage war erfolgreich, z. B. 200 OK, 201 Created, 204 No Content.

3xx – *Redirection* Der Client muss weiterspringen, z.B. 301 Moved Permanently, 302 Found, 304 Not Modified.

4xx – *Client Error* Der Client hat etwas falsch gemacht, z.B. 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed.

5xx – *Server Error* Der Server ist schuld, z.B. 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable.

Diese Einteilung werden ebenfalls in Kapitel 11 zu REST-APIs wieder aufgegriffen, da sie dort zentral benötigt werden.

1.10.4 Wichtige Headerfelder

Header sind das Arbeitstier von HTTP: Praktisch alles, was über den eigentlichen Ressourcen-Inhalt hinaus transportiert werden soll, läuft über Header. Einige wichtige Vertreter, die auch in späteren Vorlesungen vorkommen werden.

Host Pflicht ab HTTP/1.1. Gibt an, an welchen Hostnamen sich der Request richtet. Auf diese Weise kann ein einzelner Webserver mit einer einzelnen IP-Adresse mehrere Websites bedienen (*virtual hosting*).

User-Agent Identifikation des Clients (Browser, Version, Betriebssystem).

Accept, Accept-Language, Accept-Encoding Die Datenformate, Sprachen bzw. Kompressionsverfahren, die der Client verarbeiten kann. Der Server kann daraufhin seine Antwort entsprechend anpassen (*Content Negotiation*).

Content-Type Formatangabe für den Message Body, etwa `text/html; charset=UTF-8` oder `application/json`.

Content-Length Länge des Message Body in Bytes.

Set-Cookie / Cookie Server setzt bzw. Client sendet Cookies. Grundlage für Sessions (siehe Vorlesung 8).

Location Im Zusammenhang mit 3xx-Redirects: Wohin soll der Client als nächstes springen?

Cache-Control Steuert, wie und wie lange Antworten gecached werden dürfen.

1.10.5 HTTP-Versionen im Überblick

HTTP existiert seit Anfang der 1990er-Jahre und es hat sich in mehreren Schritten weiterentwickelt:

HTTP/1.0 (1996) Erste standardisierte Version. Pro Request wird eine eigene TCP-Verbindung geöffnet und nach Erhalt der Response wieder geschlossen. Bei Seiten mit vielen Ressourcen (Bilder, CSS, JS) ist das sehr ineffizient.

HTTP/1.1 (1999) Die Verbindung bleibt standardmäßig geöffnet (*Keep-Alive*), so dass mehrere Requests über dieselbe TCP-Verbindung abgewickelt werden können. Neu sind ebenfalls *Pipelining* (mehrere Requests nacheinander ohne auf die Antwort zu warten) und der verpflichtende **Host**-Header. Das ist die Version, die hier in allen Beispielen verwendet wird.

HTTP/2 (2015) Binäres Nachrichtenformat, Header-Kompression, *Multiplex* (mehrere logische Streams über eine einzige TCP-Verbindung, ohne sich gegenseitig

zu blockieren) und optionaler *Server Push*. Funktional identisch mit 1.1 (gleiche Methoden, gleiche Statuscodes), aber wesentlich effizienter.

HTTP/3 (2022) Verwendet anstelle von TCP das auf UDP basierende Protokoll *QUIC*. Das sorgt insbesondere bei verlustbehafteten Mobilverbindungen für spürbar kürzere Latenzen, weil ein verlorenes Paket nicht alle parallelen Streams ausbremst.

Für diese Einführung – also das Verständnis, wie HTTP funktioniert und wie sich HTTP-Nachrichten auf Serverseite verarbeiten lassen – ist HTTP/1.1 das richtige Abstraktionslevel. Alle höheren Versionen liefern dieselben Methoden, Statuscodes und Header; es ändert sich im Wesentlichen nur, wie diese Nachrichten konkret über die Leitung wandern. Für PHP-Skripte ist dies in der Regel nicht relevant.

1.10.6 HTTP ist zustandslos

Eine fundamentale Eigenschaft von HTTP ist, dass das Protokoll *stateless* ist. Jeder Request wird isoliert betrachtet, der Server „erinnert sich“ zunächst einmal nicht daran, dass derselbe Client vor einer Sekunde schon einmal etwas angefragt hat. Für viele Webanwendungen ist das aber nicht ausreichend – ein Online-Shop muss sich merken, was im Warenkorb liegt, und eine Login-geschützte Seite muss sich merken, wer sich angemeldet hat.

Um diesen Zustand trotzdem über mehrere Requests hinweg zu erhalten, wurden *Cookies* und darauf aufbauend *Sessions* eingeführt. Beide sind Thema von Vorlesung 8. Es benötigt aufgrund der *stateless* Eigenschaft von HTTP also zusätzliche Mechanismen, um Zustand simulieren zu können.

1.11 Der Gesamtablauf im Überblick

Mit der HTTP-Antwort hat der Client also die angeforderte Webseite erhalten, rendert sie und zeigt sie an. Damit ist der Kommunikationsablauf beendet. Was aus der User*innen-Perspektive nach einem simplen Prozess aussieht, involviert eine ganze Reihe an Protokollen und Abläufen. Abbildung 1.7 zeigt die Abfolge aller Schritte, die beim Aufruf von <https://www.fh-ooe.at/> stattfinden – vom Tastendruck bis zur fertig gerenderten Seite.

Bemerkenswert ist die Vielzahl unterschiedlicher Protokolle, die für eine scheinbar einfache Aktion – eine Webseite öffnen – zusammenspielen müssen: DHCP, DNS, ARP, IP, Ethernet, TCP, TLS und schließlich HTTP. Jedes einzelne dieser Protokolle hat eine klar umrissene Aufgabe, und erst aus diesem Zusammenspiel ergibt sich die Robustheit, mit der das Web in den letzten Jahrzehnten skaliert hat.

Ergänzend sei angemerkt: Schritt 9 umfasst tatsächlich nicht nur einen einzelnen Request, sondern eine ganze Serie. Die ursprüngliche HTML-Datei referenziert typischerweise eine Reihe weiterer Ressourcen – Stylesheets, JavaScript-Dateien, Bilder, Schriftarten, Icons. Jede dieser Ressourcen erfordert einen eigenen Request, der (dank Keep-Alive in HTTP/1.1 und Multiplexing in HTTP/2) meist über dieselbe TCP-Verbindung läuft. Der Browser folgt also iterativ den Referenzen, bis die Seite vollständig geladen und gerendert werden kann.

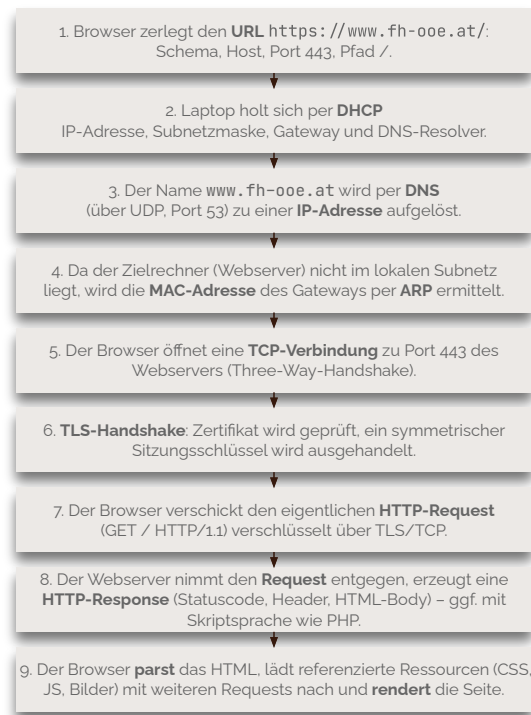


Abbildung 1.7: Der vollständige Ablauf eines Webrequests im Überblick. Jeder einzelne Schritt verwendet eines oder mehrere der in diesem Kapitel besprochenen Protokolle.

1.11.1 Brücke zur serverseitigen Entwicklung

Bis zu diesem Punkt wurde der gesamte Webrequest als Weg von A nach B betrachtet. Im weiteren Verlauf der Vorlesung steht vor allem ein einziger Schritt aus diesem Ablauf im Mittelpunkt: Schritt 8, also das, was auf dem Webserver geschieht, sobald er den Request entgegengenommen hat. In diesem Schritt wird entschieden, *was* an den Client zurückgeht: eine statische HTML-Datei aus dem Dateisystem, eine dynamisch erstellte HTML-Seite mit PHP, eine JSON-Antwort einer API, ein Redirect, eine Fehlermeldung oder auch andere Dinge sind hier möglich.

Serverseitige Programmiersprachen wie PHP setzen genau hier an. Trifft ein Request ein, dessen Pfad auf eine Datei mit der Endung `.php` verweist, übergibt der Webserver diese Datei an die PHP-Engine. Die Engine liest die Datei ein, führt den darin enthaltenen Code aus, sammelt die dabei erzeugte Ausgabe und gibt sie zusammen mit passenden HTTP-Headern als Response an den Webserver zurück. Für den Browser sieht das Ergebnis genauso aus wie bei einer statischen HTML-Datei – er erhält eine ganz normale HTTP-Response mit HTML-Body. Ob dieser HTML-Body aus einer Datei gelesen oder dynamisch erzeugt wurde, ist von außen nicht zu unterscheiden. Mehr dazu in Vorlesung 2.

1.12 Weiterführende Literatur

Für einen umfassenden Einstieg in die Netzwerk-Grundlagen bieten sich zwei Standardwerke an: [7] wählt denselben Zugang wie dieses Kapitel, es beginnt beim Anwendungsfall und arbeitet sich schichtweise nach unten. Deutschsprachige Alternativen sind [9, 12]. Alle drei Werke decken alle Abschnitte dieses Kapitels in deutlich größerer Tiefe ab und eignen sich für alle, die die Grundlagen ausführlicher nachlesen möchten.

Wer einzelne Teile des besprochenen Kommunikationsablaufs mit wunderbaren Katzenillustrationen nachlesen möchte, wird bei [1] fündig.

Für HTTP sind die RFCs die verbindliche Referenz. Zentral sind dabei RFC 9110 (*HTTP Semantics*) [4], das die semantischen Grundlagen von Methoden, Statuscodes und Headern definiert, sowie RFC 9112 (*HTTP/1.1*) [5] für die konkrete Nachrichtensyntax. Eine praxisnäher aufbereitete und laufend aktualisierte Darstellung findet sich bei MDN in [8]. Sie bietet neben Referenzmaterial auch viele kleine Beispiele zu einzelnen Headern, Methoden und Statuscodes.

Quellenverzeichnis

- [1] Article 19. *How The Internet Really Works. An Illustrated Guide to Protocols, Privacy, Censorship, and Governance*. San Francisco, CA: No Starch Press, 11. Dez. 2020 (siehe S. 1-24).
- [2] Tim Berners-Lee, Roy Fielding und Larry Masinter. *Uniform Resource Identifier (URI): Generic Syntax*. RFC 3986. IETF, Jan. 2005, S. 1–61. DOI: 10.17487/RFC3986 (siehe S. 1-3).
- [3] Tim Berners-Lee, Larry Masinter und Mark McCahill. *Uniform Resource Locators (URL)*. RFC 1738. IETF, Dez. 1994, S. 1–25. DOI: 10.17487/RFC1738 (siehe S. 1-3).
- [4] Roy T. Fielding, Mark Nottingham und Julian Reschke. *HTTP Semantics*. RFC 9110. IETF, Juni 2022, S. 1–194. DOI: 10.17487/RFC9110 (siehe S. 1-24).
- [5] Roy T. Fielding, Mark Nottingham und Julian Reschke. *HTTP/1.1*. RFC 9112. IETF, Juni 2022, S. 1–46. DOI: 10.17487/RFC9112 (siehe S. 1-24).
- [6] International Organization for Standardization. *Information technology – Open Systems Interconnection – Basic Reference Model: The Basic Model*. ISO/IEC 7498-1:1994. 15. Nov. 1994. URL: <https://www.iso.org/standard/20269.html> (siehe S. 1-7).
- [7] James F. Kurose und Keith W. Ross. *Computer Networking. A Top-Down Approach*. 8. Aufl. Harlow, Großbritannien: Pearson, 10. Juni 2021 (siehe S. 1-24).
- [8] Mozilla Developer Network. *HTTP: Hypertext Transfer Protocol*. 22. Dez. 2025. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP> (besucht am 07.04.2026) (siehe S. 1-24).
- [9] Andrew S. Tanenbaum, Nick Feamster und David J. Wetherall. *Computernetzwerke*. 6. Aufl. Hallbergmoos, Deutschland: Pearson Studium, 24. Juni 2024 (siehe S. 1-24).
- [10] W3Techs. *Usage statistics and market shares of web servers*. 6. Apr. 2026. URL: https://w3techs.com/technologies/overview/web_server (besucht am 06.04.2026) (siehe S. 1-6).
- [11] Web Hypertext Application Technology Working Group. *URL*. Living Standard. 31. März 2026. URL: <https://url.spec.whatwg.org/> (siehe S. 1-3).
- [12] Harald Zisler. *Computer-Netzwerke. Grundlagen, Funktionsweise, Anwendung*. 8. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. März 2025 (siehe S. 1-24).